

Data Structures

Dilip Maripuri
Associate Professor
Department of Computer Applications



Data Structures

Session Outline



Collision Handling

Quadratic Probing



- Quadratic probing is a collision resolution technique used in hash tables to resolve the issue of two keys hashing to the same index.
- Quadratic probing addresses collisions by trying new positions in the hash table based on a quadratic formula, rather than just moving linearly through the table.



- **Collision Resolution Process**

- **Initial Hashing:**

- A hash function $h(\text{key})$ is initially used to compute the index for a given key.

- **Collision Occurrence:**

- If the computed index is already occupied by another key (i.e., a collision occurs), quadratic probing is employed to find an alternative index.



Collision Resolution – Quadratic Probing

- Collision Resolution Process

- Probing Sequence:

- The positions are probed according to the formula

$$h(k, i) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \mod m$$

- $h(k, i)$ is the i -th probe for key k .
- $h(k)$ is the original hash function.
- i is the probe number, starting at 0.
- c_1 and c_2 are constants chosen for the probing sequence, typically $c_1 = 0$ and $c_2 = 1 \Rightarrow$
$$h(k, i) = (h(k) + i^2) \mod m$$
- m is the size of the hash table.



- Collision Resolution Process
 - Probe Until Empty Slot Found:
 - The probing continues, incrementing i , and recalculating the position until an empty slot is found or a certain condition is met (like a maximum number of probes).



- The specific choice of c_1 and c_2 for quadratic probing is designed to:
 - **Eliminate Linear Component:**
 - Setting $c_1=0$ removes the linear component of the probe sequence.
 - This means that the probe does not simply move linearly through the table but instead follows a quadratic curve.
 - This reduces the risk of primary clustering, a common issue with linear probing where clusters of entries develop, leading to longer search times.



Collision Resolution – Quadratic Probing



- The specific choice of c_1 and c_2 for quadratic probing is designed to:
 - **Emphasize Quadratic Growth:**
 - Setting $c_2 = 1$ ensures that the probing sequence emphasizes the quadratic component, where the increment to the next probe position grows quadratically rather than linearly.
 - This spreads out the probe locations more effectively over the hash table.



- Characteristics
 - Secondary Clustering:
 - Unlike linear probing, which suffers from primary clustering where consecutive elements form groups, quadratic probing reduces this effect but can still experience secondary clustering.



- **Characteristics**
 - **Efficient Memory Utilization:**
 - It utilizes the memory better than some other methods (like chaining) as it stores all elements directly in the hash table without requiring additional data structures.



- **Limitations**

- **Looping:**

- A badly configured probe sequence can lead to a situation where the sequence loops back to already tried slots, leading to infinite loops if not correctly managed.

- **Hash Table Size (m):** 11
- **Hash Function:** $h(\text{key}) = \text{key} \% 11$
- **Quadratic Probing Formula:** $h(k, i) = (h(k) + i^2) \% 11$
- **Keys to be Inserted:** [14, 28, 22, 32, 41, 58, 62, 89, 17, 99, 45]



Collision Resolution – Quadratic Probing

Step	Key	$h(\text{key}) / (\text{Initial Index})$	Collision Resolution	Final Index	Hash Table Content after Insertion
1	14	$14 \% 11 = 3$	None	3	[_, _, _, 14, _, _, _, _, _, _, _]
2	28	$28 \% 11 = 6$	None	6	[_, _, _, 14, _, _, 28, _, _, _, _]
3	22	$22 \% 11 = 0$	None	0	[22, _, _, 14, _, _, 28, _, _, _, _]
4	32	$32 \% 11 = 10$	None	10	[22, _, _, 14, _, _, 28, _, _, _, 32]
5	41	$41 \% 11 = 8$	None	8	[22, _, _, 14, _, _, 28, _, 41, _, 32]
6	58	$58 \% 11 = 3$	$3 + 1^2 = 4$	4	[22, _, _, 14, 58, _, 28, _, 41, _, 32]



Collision Resolution – Quadratic Probing



Step	Key	$h(\text{key}) / (\text{Initial Index})$	Collision Resolution	Final Index	Hash Table Content after Insertion
7	62	$62\%11=7$	None	7	[22, _, _, 14, 58, _, 28, 62, 41, _, 32]
8	89	$89\%11=1$	None	1	[22, 89, _, 14, 58, _, 28, 62, 41, _, 32]
9	17	$17\%11=6$	$(6+1^2)\%11=7,$ $(6+2^2)\%11=10,$ $(6+3^2)\%11=4,$ $(6+4^2)\%11=0,$ $(6+5^2)\%11=9$	9	[22, 89, _, 14, 58, _, 28, 62, 41, 17, 32]
10	99	$99\%11=0$	$(0+1^2)\%11=1$ $(0+2^2)\%11=4$ $(0+3^2)\%11=9$ $(0+4^2)\%11=5$	5	[22, 89, _, 14, 58, 99, 28, 62, 41, 17, 32]
11	45	$45\%11=1$	$(1+1^2)\%11=2$	2	[22, 89, 45, 14, 58, 99, 28, 62, 41, 17, 32]



Collision Resolution – Quadratic Probing

Step	Key	Initial Index	Probing Sequence	Probed Index	Value at Index	Found?
1	45	1	$i = 0$ (No probe)	1	89	No
2	45	1	$i = 1$ (Probe 1) $(1+1^2)\%11=2$	2	45	No



- When performing a search using quadratic probing in a hash table, encountering an empty slot can significantly impact the outcome of the search.
- Specifically, if you're searching for a key and you encounter an empty slot, this indicates that the key does not exist in the table.
- This happens because quadratic probing fills in any found empty slots during insertion, so any subsequent searches that hit an empty slot mean the key was never placed in the table after that slot was last emptied.

Collision Resolution – Quadratic Probing

- Size: 11
- Hash Function: $h(\text{key}) = \text{key} \% 11$
- Quadratic Probing Formula: $h(k, i) = (h(k) + i^2) \% 11$
- Keys Inserted: [14, 28, 22, 32, 41, 58, 62, 89, 17, 99]
- Final Hash Table State: [22, 89, -, 14, 58, 99, 28, 62, 41, 17, 32]



Collision Resolution – Quadratic Probing

- Final Hash Table State: [22, 89, -, 14, 58, 99, 28, 62, 41, 17, 32]

Step	Key	Initial Index	Probing Sequence	Probed Index	Value at Index	Found?	Termination Condition
1	70	4	$i = 0$ (No probe)	4	58	No	Value not match
2	70	4	$i = 1$ (Probe 1)	5	99	No	Value not match
3	70	4	$i = 2$ (Probe 2)	8	41	No	Value not match
4	70	4	$i = 3$ (Probe 3)	2	-	No	Empty slot found



Double Hashing



- Double hashing is a method of collision resolution in hash tables that utilizes two hash functions to calculate the position of an entry, providing a way to systematically probe for alternative slots.



- **Probing Sequence:**

- When a collision occurs, the next index is calculated using a combination of the two hash functions:

$$(h1(key) + i \cdot h2(key)) \mod m$$

where i is the probe number (0, 1, 2, ...), and m is the size of the hash table.

- It's important that $h_2(\text{key})$ is relatively prime to the m - Why?
 - **Full Coverage of Hash Table:**
 - When $h_2(\text{key})$ is relatively prime to m , the probing sequence generated will cover every slot in the hash table over a complete sequence of probes.
 - This is because the step interval $h_2(\text{key})$ will vary sufficiently so that every subsequent probe position is distinct from the previous ones until all slots have been tried.



Data Structures

Collision Resolution – Double Hashing



- It's important that $h_2(\text{key})$ is relatively prime to the m - Why?
 - **Avoiding Infinite Loops:**
 - If $h_2(\text{key})$ is not relatively prime to m , then the probing sequence might repeat itself before all slots have been checked.
 - For example, if $m=10$ and $h_2(\text{key})=5$, the probing sequence (when you add $h_2(\text{key}) \times i$ for $i=0,1,2,\dots$) will only yield indices that are either all odd or all even, never both.
 - This means you could end up exploring only half of the hash table slots in the worst case, potentially missing the empty slot needed to insert the new key.

- It's important that $h_2(\text{key})$ is relatively prime to the m - Why?
 - Ensuring Efficiency:
 - Having a relatively prime $h_2(\text{key})$ ensures that the probes are efficiently distributed across the hash table, minimizing the likelihood of needing to probe many times before finding an empty slot or returning to the start of a cycle.

- **How to formulate $h_2(\text{key})$**
 - **Subtractive Method:**
 - A simple and popular choice is to subtract the key modulus some number from another number, ensuring the result is never zero.
 - **Example**
 - $h_2(\text{key}) = 7 - (\text{key} \% 7)$ (adjust numbers to ensure non-zero and relative primeness)

- **Hash Table Size (m):** 13 (a prime number to facilitate good distribution)
- **Initial Hash Function h1:** $h1(key) = key \% 13$
- **Secondary Hash Function h2:** $h2(key) = 7 - (key \% 7)$
- **Keys:** [18, 41, 64, 36, 92, 59, 32]



Data Structures



Collision Resolution - Double Hashing

Hash Table Size (M)

13

Hash Function (H(Key))

Key mod M

Second Hash Function

7-(key Mod 7)

Calculated Index

$(h(\text{key}) + i * h_2(\text{Key})) \bmod M$

18, 41, 64, 36, 92, 59, 32, 15, 28, 41

	0	1	2	3	4	5	6	7	8	9	10	11	12
		92	41	41		18	32	59	15	28	36		64
Step No	Key	h1(key)	h2(key)	Attempt / Probe	i	Calculated Index	Action						
1	18	5	3	1	0	5	NO						
2	41	2	1	1	0	2	NO						
3	64	12	6	1	0	12	NO						
4	36	10	6	1	0	10	NO						
5	92	1	6	1	0	1	NO						
6	59	7	4	1	0	7	NO						
7	32	6	3	1	0	6	NO						
8	15	2	6	1	0	2	YES						
	15	2	6	2	1	8	NO						
9	28	2	7	1	0	2	YES						
	28	2	7	2	1	9	NO						
10	41	2	1	1	0	2	YES						
	41	2	1	2	1	3	NO						

Data	Total Number of Probes taken to insert
18	1
41	1
64	1
36	1
92	1
59	1
32	1
15	2
28	2
41	2



Collision Resolution - Double Hashing



Feature/Method	Linear Probing	Quadratic Probing	Double Hashing
Initial Probe	$h(key) = key \% m$	$h(key) = key \% m$	$h_1(key) = key \% m$
Probing Sequence	Linearly increments: $h(key, i) = (h(key) + i) \% m$	Quadratically increments: $h(key, i) = (h(key) + i^2) \% m$	Varied increments: $h(key, i) = (h_1(key) + i \cdot h_2(key)) \% m$
Clustering	High primary clustering	Lower primary, some secondary clustering	Minimal primary and secondary clustering
Performance at High Load Factor	Performance degrades due to clustering	Better than linear due to reduced clustering	Maintains performance longer due to more uniform probing
Ease of Implementation	Simplest to implement	More complex than linear but manageable	Most complex due to the need for two hash functions
Probe Efficiency	May need many probes in dense areas	Improved spread but can revisit areas	Highly efficient probing, typically visits each slot
Table Utilization	Can lead to long probe sequences in full tables	Better space utilization due to spread	Best utilization, all slots are reachable
Typical Use Case	Good for tables with low load factors	Suitable for moderately high load factors	Excellent for tables with high load factors and where consistent performance is critical



THANK YOU

Dilip Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu